

An Incremental Exact Solver for the Minimum Line Cover of Prime Points

Jesper Gran Mikkelsen
Independent Researcher, Norway

April 2026

Abstract

Given the N prime points $(1, p_1), (2, p_2), \dots, (N, p_N)$ in the plane, how many straight lines suffice to pass through all of them simultaneously? The answer forms OEIS sequence A373813; its terms are elementary to state yet resistant to computation, because every useful line is an arithmetic coincidence among prime values and large instances offer vanishingly few such coincidences to exploit.

The prior certified record stood at $N = 861$, obtained by industrial mixed-integer programming solvers after 282 hours, 26 minutes, and 31.5 seconds of wall-clock time on dedicated hardware. I report the new record $N = 882$, certified optimal. The full sweep from $N = 1$ through $N = 882$ completes in approximately 14 hours on a standard consumer laptop; the prior record boundary is surpassed—that is, $N = 862$ is certified—in approximately three hours.

The solver described here, `primecover1024.cpp`, enumerates all heavy lines once for the entire run horizon, rewrites the objective as heavy-line gain maximisation, and sweeps incrementally through successive values of N . At each step it carries a witness cover, a warm heavy-line primal seed, and a warm Lagrangian dual seed from the previous prefix, launching full exact search only when the carried witness fails to cover the new prime point. The exact phase combines a root cover-inequality bound, a Lagrangian gain bound with projected subgradient ascent and coordinate-descent polish, the Exclusive Dependency Rule, strong branching on a shortlist of at most four candidates, and a parallel frontier of independent subproblems. On the same hardware, the incremental sweep reaches the prior record boundary $N = 861$ in 13,115 seconds—approximately three hours—compared with 282 hours for the prior approach at that same boundary.

The computation extends A373813 by 21 new terms ($N = 862$ through $N = 882$), including three new awkward primes—values of N at which the minimum line count increases—occurring at $N = 864, 871$, and 875 . For every N at which the solver terminates before any per- N time limit fires, the reported answer is provably optimal: the algorithm closes the gap between its lower bound and its best incumbent to zero before halting.

MSC 2020 Subject Classification: 05B40, 11A41, 68W25, 68W40.

Keywords: prime points, minimum line cover, exact search, Lagrangian relaxation, witness propagation, OEIS A373813.

1 Introduction

The problem

Given the N points $(1, p_1), (2, p_2), \dots, (N, p_N)$ in the plane, where p_i denotes the i -th prime, find the minimum number of straight lines required to pass through all N points simultaneously. I write

$$S_N := \{(1, p_1), (2, p_2), \dots, (N, p_N)\}.$$

The statement is elementary; the difficulty is not. Three or more points of $\mathcal{S}_N$ are collinear only when their prime values satisfy an arithmetic relationship that is simultaneously exact enough to land on a line and rich enough to involve more than two points. As N grows, such coincidences become rarer relative to the size of the instance, and any formulation that does not identify and exploit them structurally pays a severe computational penalty.

The minimum cover numbers $f(N)$ form OEIS sequence A373813 [1]. The sequence is non-decreasing and mostly flat: $f(N) = f(N - 1)$ at all but finitely many indices in any finite prefix, because almost every new prime point lies on an existing cover line. The exceptional indices—those at which $f(N) > f(N - 1)$ —are the *awkward primes*. Certifying a new record requires proving optimality at every awkward prime up to and including the current one.

Prior work

The prior certified record stood at $N = 861$ [2], obtained using industrial mixed-integer programming solvers after 282 hours, 26 minutes, and 31.5 seconds of wall-clock time on dedicated hardware. That figure covers the computation for $N = 861$ alone; the solver treated each prefix independently, relied on a general-purpose MIP engine with no problem-specific warm-starting, and scaled poorly with N precisely because it could not carry any structural insight from one instance to the next.

Contribution

I replace the MIP approach with `primecover1024.cpp`, a purpose-built exact solver that is incremental by design. The program works as follows. It first enumerates all lines passing through at least three points of $\mathcal{S}_{N_{\text{end}}}$ (the *heavy lines*) once for the entire run horizon N_{end} . It then sweeps through $N = 1, 2, \dots, N_{\text{end}}$, maintaining at each step a *witness cover*—a feasible set of lines for the current prefix—together with a warm primal seed and a warm Lagrangian dual seed. When the new prime p_N already lies on a line in the current witness, the result is inherited in constant time. Only when the witness fails does the solver launch full exact search, which it initialises from the carried warm state rather than from scratch.

The exact search combines a root cover-inequality lower bound, a Lagrangian gain bound tightened by projected subgradient ascent and coordinate-descent polish, the Exclusive Dependency Rule for structural pruning, strong branching on a shortlist of at most four candidates, and a parallel frontier of independent subproblems processed concurrently. Every node closes to certifiable gap zero before the search halts; no heuristic termination is possible.

The concrete outcomes are as follows.

- **New record.** $f(882) = 126$, certified optimal. The per-instance solve time for $N = 882$ is 1,092.070 seconds (approximately 18 minutes) on a standard consumer laptop.
- **Incremental sweep.** The cumulative sweep from $N = 1$ through $N = 862$ —surpassing the prior record—completes in approximately three hours (13,115 seconds) on a standard consumer laptop, compared with 282 hours on dedicated hardware for the prior computation at that boundary. The full sweep through $N = 882$ completes in approximately 14 hours (51,256 seconds) on the same machine.
- **Sequence extension.** A373813 is extended by 21 new certified terms ($N = 862$ through $N = 882$), including three new awkward primes at $N = 864, 871$, and 875 .
- **Optimality guarantee.** For every N at which the solver terminates before any per- N time limit fires, the reported $f(N)$ is provably optimal. The algorithm closes the primal–dual gap to zero before halting; it cannot report a value it has not certified.

The remainder of the paper follows the program. [Section 2](#) defines the geometric objects, the gain reformulation, and the task state. [Section 3](#) describes the two-phase structure, including heavy-line enumeration, the exact-search skeleton, and the parallel frontier mechanism. [Section 4](#) states the three execution modes exactly and describes witness construction, witness reuse, and the state carried between successive values of N . [Section 5](#) treats the cover inequality, the Lagrangian bound, the projected subgradient and polish steps, and the branch-specific child caps. [Section 6](#) proves correctness, including the Exclusive Dependency Rule and the mode-by-mode certification guarantee. [Section 7](#) records the new OEIS data and the per-instance statistics extracted from the preserved sweep log. [Section 8](#) closes with code-availability notes and the precise meaning of “1024” in the filename.

2 Problem Definition

2.1 Prime Points and Heavy Lines

I fix a run horizon $N_{\text{end}} \geq 1$ and write

$$[N] := \{1, 2, \dots, N\}.$$

The driver computes the first N_{end} primes and works with

$$\mathcal{S}_N = \{(i, p_i) : 1 \leq i \leq N\}, \quad 1 \leq N \leq N_{\text{end}}.$$

The implementation stores point indices as $0, 1, \dots, N - 1$. I keep the mathematical notation $1, \dots, N$ and translate to 0-based indices only when I mirror a code field exactly.

Definition 2.1 (Minimum line cover number). *I write $f(N)$ for the minimum cardinality of a family of straight lines whose union contains $\mathcal{S}_N$.*

Two milestone covers make the geometry concrete. [Figure 1](#) shows the final prefix that two lines can cover, while [figure 2](#) shows the final prefix with $f(N) = 5$. In the latter case the five-line cover is

$$\{(1, 2), (24, 89)\}, \quad \{(2, 3), (3, 5)\}, \quad \{(4, 7), (5, 11), (17, 59), (19, 67), (20, 71), (22, 79), (23, 83)\}, \\ \{(6, 13), (7, 17), (10, 29), (12, 37), (13, 41), (16, 53), (18, 61), (21, 73)\}, \quad \{(8, 19), (9, 23), (11, 31), (14, 43), (15, 47)\}.$$

Here I display the points that each chosen line is needed to support; the coral line in [figure 2](#) also passes through $(4, 7)$, but that incidence is redundant because the olive line already covers $(4, 7)$. These two pictures already exhibit the plateau structure that later awkward primes interrupt.

Definition 2.2 (Canonical line). *Given two distinct points (x_1, y_1) and (x_2, y_2) , I write*

$$a = y_1 - y_2, \quad b = x_2 - x_1, \quad c = -(ax_1 + by_1).$$

I then divide (a, b, c) by $\gcd(|a|, |b|, |c|)$ and, if necessary, multiply by -1 so that either $a > 0$ or $a = 0$ and $b > 0$. The resulting primitive triple defines the canonical equation

$$ax + by + c = 0.$$

This is exactly the normalization used by `canonical_line()`.

Definition 2.3 (Heavy line). *A line is heavy if it contains at least three points of $\mathcal{S}_{N_{\text{end}}}$. I denote the set of all heavy lines by $\mathcal{H}$. For $h \in \mathcal{H}$, I write*

$$\text{pts}(h) = \{i_1 < i_2 < \dots < i_t\} \subseteq [N_{\text{end}}]$$

for its point indices.

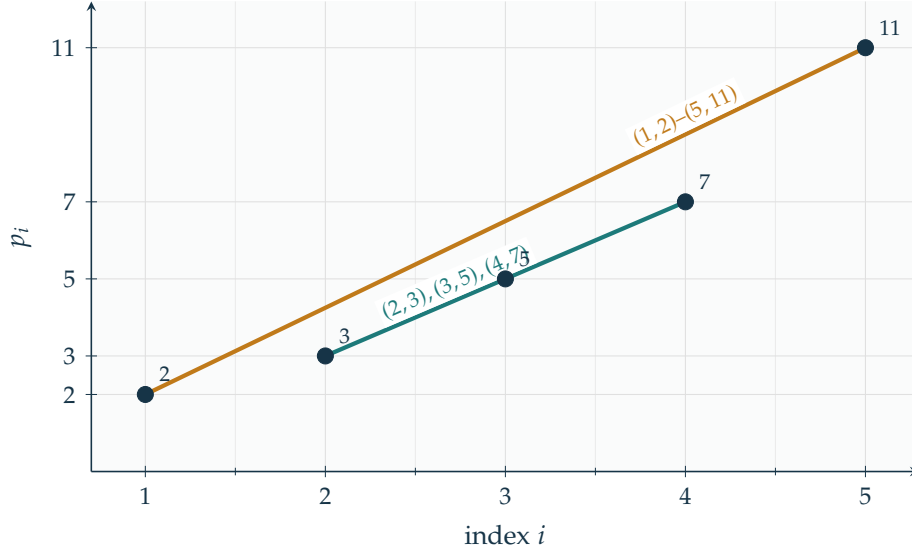


Figure 1: The prefix $\mathcal{S}_5$ is the last instance coverable by only two lines. One support line joins (1, 2) to (5, 11), while the second line captures the three-point coincidence (2, 3), (3, 5), (4, 7). Hence $f(5) = 2$, but the next prime already forces $f(6) = 3$.

Definition 2.4 (Activation index). For $h \in \mathcal{H}$ with $\text{pts}(h) = \{i_1 < i_2 < \dots < i_t\}$ (indices in 1-based notation), I define

$$\text{act}(h) := i_3.$$

At step N , the active heavy lines are

$$A_N := \{h \in \mathcal{H} : \text{act}(h) \leq N\}.$$

The activation rule matches the code exactly: the implementation stores `activate_at = h1.points[2] + 1` (converting from 0-based to 1-based), and a heavy line enters `active_ids` at sweep step $N = \text{act}(h)$. A line becomes active at the step whose index equals its third point, which is the first value of N for which $\mathcal{S}_N$ already contains all three of the line's defining points.

2.2 Gain Reformulation

For $H \subseteq A_N$, I write

$$\text{cov}_N(H) := \bigcup_{h \in H} (\text{pts}(h) \cap [N]).$$

After I select the heavy lines in H , the uncovered points can be finished with arbitrary 2-point lines and, if needed, one singleton line. The solver therefore optimizes a gain objective rather than the cover size directly.

Definition 2.5 (Gain). For $H \subseteq A_N$, I define

$$\text{gain}_N(H) := |\text{cov}_N(H)| - 2|H|.$$

Proposition 2.6 (Cost-gain equivalence). For every $H \subseteq A_N$,

$$|H| + \left\lceil \frac{N - |\text{cov}_N(H)|}{2} \right\rceil = \left\lceil \frac{N - \text{gain}_N(H)}{2} \right\rceil.$$

Hence minimizing the number of lines is equivalent to maximizing $\text{gain}_N(H)$.

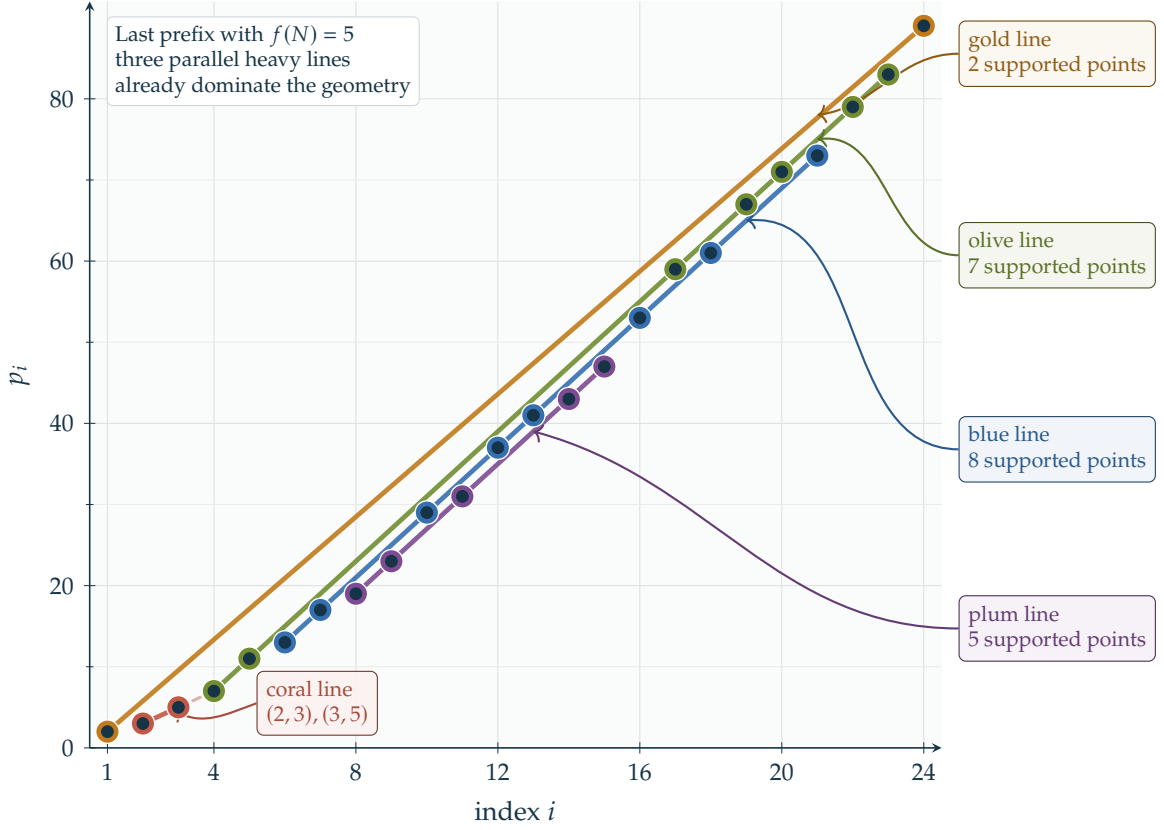


Figure 2: The prefix S_{24} is the final instance coverable by five lines. The long olive, blue, and plum segments are three parallel heavy lines of slope 4; the coral segment is the auxiliary line chosen for the pair $(2, 3), (3, 5)$; and the gold segment is the remaining two-point support line from $(1, 2)$ to $(24, 89)$. The faint dashed coral continuation indicates that the same geometric line also passes through $(4, 7)$, but that point is already covered by the olive line. After $N = 24$, a sixth line becomes unavoidable.

Proof. If I fix the heavy lines in H , then exactly $N - |\text{cov}_N(H)|$ points remain uncovered. Any additional line can cover at most two of them, and any two uncovered points determine a line, so the optimal residual cost is $\lceil (N - |\text{cov}_N(H)|)/2 \rceil$. Adding the $|H|$ heavy lines gives the left-hand side. The right-hand side is the same quantity after I substitute the definition of $\text{gain}_N(H)$. $\square$

2.3 Search Nodes

The exact solver searches over subsets of active heavy lines. At a node T , it stores:

- the active point set $U(T) \subseteq [N]$;
- for every active heavy line, its residual cover count

$$c_T(h) := |\text{pts}(h) \cap U(T)|;$$

- the currently available heavy lines;
- the selected heavy lines, stored in `current_choice`; and
- the accumulated gain, stored in `current_gain`.

Definition 2.7 (Productive line). *At node T , an available heavy line h is productive if $c_T(h) \geq 3$.*

Algorithm 1: ENUMERATEHEAVYLINES($p_1, \dots, p_{N_{\text{end}}}$)

```
1 for  $i \leftarrow 1$  to  $N_{\text{end}}$  do
2   clear all slope buckets
3   for  $j \leftarrow i + 1$  to  $N_{\text{end}}$  do
4     compute the reduced slope of  $(i, p_i)$  and  $(j, p_j)$ 
5     append  $j$  to the corresponding bucket
6   foreach bucket  $B$  with  $|B| \geq 2$  do
7     if  $i$  is the leftmost anchor on that slope then
8       emit the line through  $\{i\} \cup B$ 
9       set its activation index to the third smallest point of  $\{i\} \cup B$ 
10 sort the emitted lines by first point, then by decreasing size, then lexicographically by point list
```

If the solver selects a productive line h at node T , the gain increases by

$$\delta_T(h) := c_T(h) - 2.$$

This matches the code path `positive_gain(task, id)`. A line with residual coverage at most 2 never contributes positive gain again, because the active point set only shrinks.

The implementation also records the sorted list

$$B(T) := \{\text{unavailable heavy lines with residual coverage at least 3}\},$$

because that list appears in frontier-state dominance. Any unavailable line with residual coverage at most 2 is omitted from $B(T)$, since such a line can never become productive again once the active set only shrinks.

3 Algorithm Overview

3.1 Phase 1: Heavy-Line Enumeration

The program first generates the first N_{end} primes and enumerates every heavy line in $\mathcal{S}_{N_{\text{end}}}$. For each anchor point i , it groups later points $j > i$ by reduced slope. A bucket with at least two later points gives a candidate heavy line, and the routine keeps that line only when i is the leftmost point on the same arithmetic progression. This leftmost-anchor test is the program's only de-duplication rule.

After enumeration, the driver constructs two lookup structures that the sweep reuses:

- an incidence list that maps each point to the heavy lines containing it; and
- an activation list that maps each N to the heavy lines with activation index N .

3.2 Phase 2: Incremental Sweep

The sweep then processes $N = 1, 2, \dots, N_{\text{end}}$ in order. At step N , it appends the newly activated heavy lines to `active_ids`. It also computes a purely greedy cover, `greedy_upper_bound_solution()`, by repeatedly selecting the active heavy line that covers the most currently uncovered points, provided that count exceeds 2 (i.e., the line must cover at least three uncovered points). When two lines tie on uncovered-point count, the code prefers the one with more total points in its point list; when that tie persists as well, it retains the first such line encountered in the iteration order of `active_ids`. After no heavy line can contribute more than two newly covered points, the routine closes the residual with $\lceil r/2 \rceil$

pair lines, where r is the number of uncovered points remaining. The resulting cost is the logged quantity `ub_raw`.

The driver also logs

$$\text{ub0} = \min\{N, \text{prev} + 1, \text{ub_raw}\},$$

where `prev` is the last certified answer, and it tightens this diagnostic to $\min\{\text{ub0}, \text{prev}\}$ when the witness already covers (N, p_N) . The source uses `ub0` only as a sweep-level diagnostic. The exact solver does *not* import `ub0` as a scalar incumbent; instead, it seeds the search with explicit heavy-line sets.

The only scalar lower bound that the driver passes into the exact solver is

$$\text{lb0_floor} = \begin{cases} \text{prev}, & \text{if warm_start_solution is nonempty,} \\ 0, & \text{otherwise.} \end{cases}$$

The `solve_exact()` wrapper combines this floor with the cover inequality lower bound before calling the gain solver: the effective floor passed to `solve_for_n` is $\max\{\text{lb0_floor}, \text{lb}_{\text{cov}}\}$. Inside the gain solver, `capture_root_cost_lb()` maximizes this combined floor against the first captured Lagrangian cost bound to produce the final root lower bound `lb` (see [section 5](#)).

3.3 Exact Search and the Parallel Frontier

On a witness miss, the program calls `solve_exact()`. The exact solver constructs the root task and then prepares an initial incumbent before building expensive bounds. The seeding sequence, in order, is:

1. **Candidate from warm start.** The solver copies the root task, replays `warm_start_solution` (skipping any line with nonpositive residual gain), and submits the resulting gain as a candidate. The original root task is not modified.
2. **Candidate from greedy seed.** The solver copies the root task, replays the greedy heavy-line set `greedy_seed_line_ids`, and submits that gain as a candidate. Again the original root is unchanged.
3. **Root forcing.** The solver applies the Exclusive Dependency Rule to the root task itself via `apply_root_forced_line_reduction()`, permanently forcing any provably includable productive line into the partial solution.
4. **Greedy completion from forced root.** A greedy completion is run on the forced root state and submitted if it beats the current incumbent.
5. **Greedy completion from warm start.** A greedy completion is run starting from the warm-start lines applied to the forced root, and submitted if it beats the incumbent.

After step 3, the solver records the productive-line count `prod` and total productive incidence `pinc` at the forced root state. These are the values printed in the sweep log and tabulated in [section 7](#).

The two greedy routines are distinct. The driver-level `greedy_upper_bound_solution()` is fully adaptive: after each selected heavy line it recomputes the best next candidate from scratch across all active lines. The exact solver's internal `greedy_completion()` is cheaper: it buckets productive lines by their initial residual gain, scans those buckets from high to low, recomputes each line's residual gain against a running temporary mask just before use, selects it if that recomputed gain is still positive, and never re-buckets remaining lines after any selection.

If the machine reports a single hardware thread, the solver runs depth-first search on the forced root task directly. If it reports multiple threads, it first builds a parallel frontier whose

target size is

$$\text{workers} \times \begin{cases} 128, & \text{if the current best cost is at least 113,} \\ 16, & \text{if the current best cost is at least 93,} \\ 4, & \text{otherwise.} \end{cases}$$

To expand the frontier, the code repeatedly extracts the task with largest active-point count, breaking ties in favor of smaller current gain. It applies the root forcing rule again, computes the Lagrangian bound, branches on one productive line, and inserts every surviving child into a dominance map keyed by $(U(T), B(T))$.

The exact search itself branches only on productive heavy lines. The branch selector first forms a shortlist of up to four productive candidates. It orders them by larger residual gain $c_T(h) - 2$, then by larger nonnegative Lagrangian slack, then by larger

$$\sum_{p \in \text{pts}(h) \cap U(T)} |\text{ordered_point_lines}[p]|,$$

which the code uses as a branch-pressure surrogate, and finally by smaller line id. For each shortlisted line, the solver computes the exclude-child cap described in [section 5](#). If the shortlist has more than one line, the node cost lower bound is within 1 of the current incumbent cost, $|U(T)| \geq 64$, and the current partial choice satisfies $|\text{current_choice}| \leq 6$, then the code performs strong branching: it tentatively evaluates each include child with at most 24 Lagrangian iterations and chooses the line that prunes the most children, then minimizes the worse child bound, then the sum of child bounds, then reverts to the same ranking rule. Here $|\text{current_choice}|$ is the number of already selected heavy lines, including any lines forced earlier on the same path. Otherwise the solver simply branches on the top-ranked shortlist line.

The dominance key is

$$(U(T), B(T)),$$

where $B(T)$ is the sorted list of unavailable lines that still have residual coverage at least 3. The solver keeps only the largest current gain seen for each key. After the frontier is ready, worker threads pull tasks through an atomic `fetch_add` counter. Each worker keeps its dual vectors, greedy buckets, undo stacks, and scratch buffers locally; the only shared search state is the atomic incumbent gain and the frontier counter.

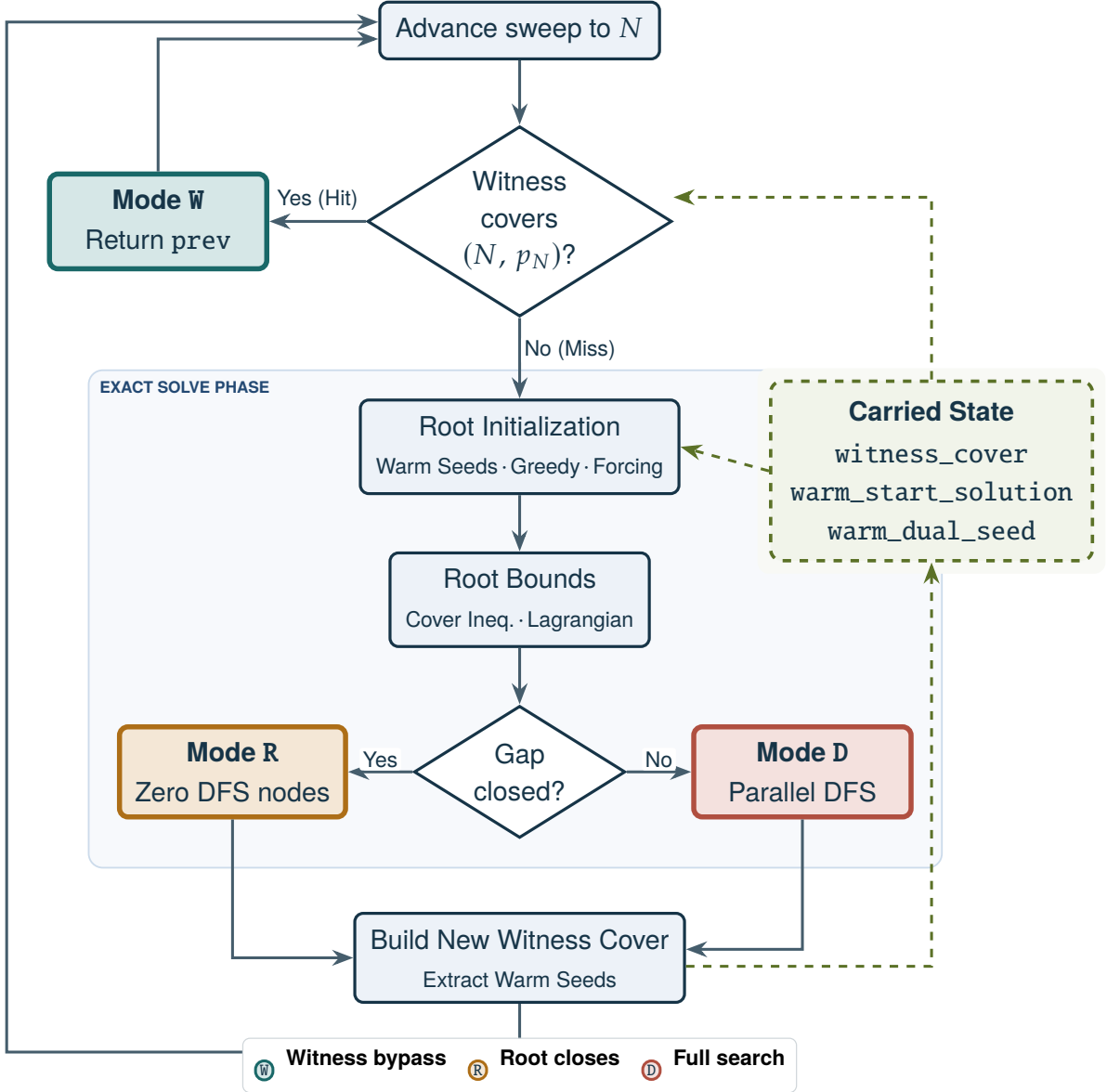


Figure 3: Algorithm architecture and state propagation across the incremental sweep. At each step N , the solver evaluates the carried `witness_cover`. A hit (Mode W) bypasses exact search entirely and returns `prev` immediately. On a miss, the solver enters the *exact solve phase*: it initialises the root node with the carried `warm_start_solution` and `warm_dual_seed`, tightens root bounds via the cover inequality and the Lagrangian bound, and then branches. If the bounds close the gap before any DFS node is counted, the mode is R; otherwise the parallel frontier is launched (Mode D). Both exact modes conclude by rebuilding the witness and extracting new warm seeds to carry into the next iteration. Dashed arrows show the three carried-state channels.

4 Incremental Warm-Start and Witness Propagation

4.1 Carried State and the Three Modes

The incremental structure is the main design feature of the program. Between successive values of N , the driver carries:

- `prev`, the most recent certified cover size;
- `active_ids`, the current active heavy lines;
- `warm_start_solution`, the heavy lines returned by the most recent successful call to `solve_exact()`;
- `warm_dual_seed`, the most recently exported nonempty dual seed; and
- `witness_cover` together with the Boolean `witness_ready`.

The driver distinguishes three modes exactly as follows.

Mode W (witness hit). The driver enters mode W if and only if `witness_ready` is true and `witness_covers_point(witness_cover, N, p_N)` returns true. In that case it returns `prev` immediately and does not call `solve_exact()`. The carried witness, warm heavy-line set, and warm dual seed therefore remain unchanged.

Mode R (exact solve with zero DFS nodes). If the witness misses, the driver calls `solve_exact()`. It records mode R exactly when the returned field nodes is zero. This is not a different algorithmic path: it is the case where seeding, root forcing, greedy completion, and the first captured lower bound already close the root interval before the DFS node counter ever increments.

Mode D (exact solve with positive DFS count). If the witness misses and the returned field nodes is positive, the driver records mode D.

In the supplied full-sweep log `PRIMECOVER_WR_882.txt`, the prefix $N = 1, \dots, 882$ contains 470 witness hits, 286 mode-R rows, and 126 mode-D rows. On the new block $N = 862, \dots, 882$, the same split is 5 witness hits, 5 mode-R rows, and 11 mode-D rows.

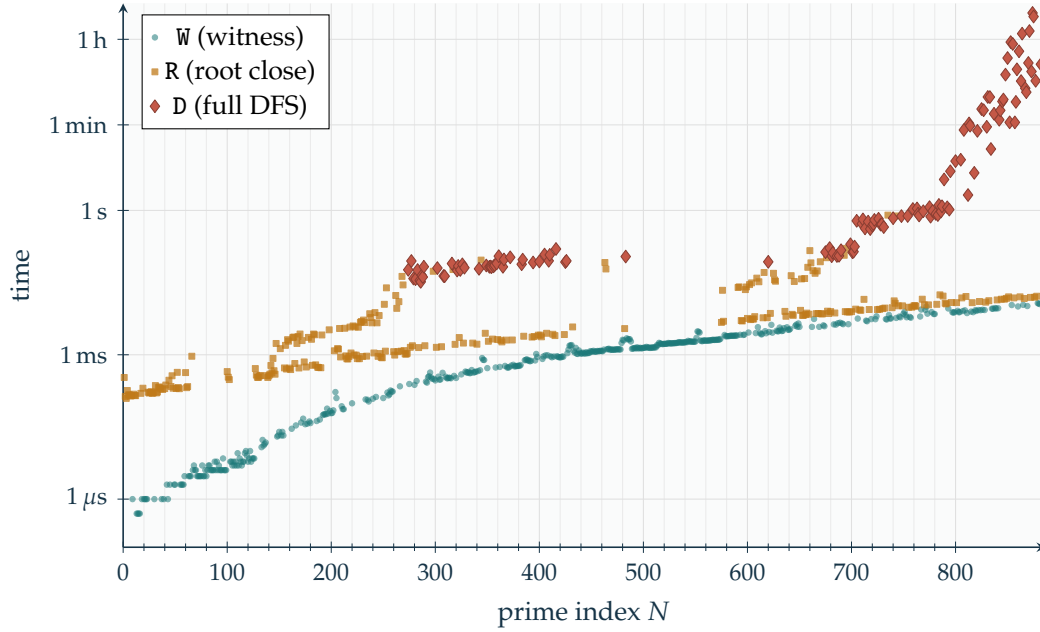


Figure 4: Per-instance solve time on a logarithmic scale across all 882 certified indices, colour-coded by execution mode. Teal circles (W) are witness hits; gold squares (R) are root closures requiring no DFS; coral diamonds (D) are full depth-first searches. The seven-order-of-magnitude spread—from sub-microsecond witness hits to hours-long DFS runs—makes the witness mechanism’s value immediately visible: 470 near-zero teal circles sit alongside a coral diamond at $N = 874$ reaching 12,683 seconds (≈ 3.5 h). The five highest coral diamonds ($N = 874, 875, 871, 864, 853$) account for 71.5 % of all compute time across the full sweep.

4.2 Witness Construction

After every successful call to `solve_exact()`, the driver builds a witness cover by calling

`build_witness_cover( $N, N_{\text{end}}, \text{primes}, \text{lines}, \text{warm\_start\_solution}$ ).`

The witness horizon is therefore the final requested run limit, not merely $N + 1$.

The construction is deterministic.

1. For each chosen heavy line, the code writes down the canonical line through its first two points. That line already covers every point of the heavy line.
2. It marks every point of $\mathcal{S}_N$ already covered by those heavy lines and lists the uncovered indices in increasing order.
3. If $N < N_{\text{end}}$ and at least two residual points remain, it scans future indices $t = N + 1, N + 2, \dots$ until it finds the first future point not yet covered by the witness assembled so far.
4. Among all unused residual pairs whose line passes through (t, p_t) , it chooses the pair that maximizes:
 - first, the *reach*, namely the largest number of consecutive future indices starting at t for which each point is either already covered or lies on the candidate line;
 - second, the total number of not-yet-covered future points on the candidate line from t through N_{end} ; and
 - third, lexicographically by the residual pair (smaller first residual index, then smaller second residual index).

5. It repeats the previous step until either every future point up to N_{end} is covered or no residual pair passes through the next uncovered future point.
6. It then pairs the remaining residual points in increasing order. If one residual point is left over, the code adds the vertical line $x = i$ through that point.

Although the future-aware pairing is heuristic, the witness itself is exact for the current prefix.

Proposition 4.1. *Suppose `chosen_line_ids` is any heavy-line set for $\mathcal{S}_N$. Then `build_witness_cover` returns a genuine cover of $\mathcal{S}_N$ with*

$$|\text{chosen_line_ids}| + \left\lceil \frac{r}{2} \right\rceil$$

lines, where r is the number of points of $\mathcal{S}_N$ not covered by those heavy lines.

Proof. Each chosen heavy line contributes one exact geometric line and covers precisely the points the solver marks. The remaining uncovered points of $\mathcal{S}_N$ form the residual list. Every subsequent line that the routine adds uses either two residual points or, in the odd case, one residual point together with a vertical singleton line. Hence every residual point is covered and the total number of residual lines is exactly $\lceil r/2 \rceil$. $\square$

If `warm_start_solution` is optimal for $\mathcal{S}_N$, then [theorem 4.1](#) and [theorem 2.6](#) show that the witness has exactly $f(N)$ lines.

4.3 Warm Heavy-Line Seeds and Warm Dual Seeds

The warm start has two distinct components.

First, the solver carries the heavy lines chosen by the most recent successful exact solve. On the next witness miss, the exact solver replays that set at the new value of N , discarding any line whose current residual gain is nonpositive. It also uses the same set as the base for one extra greedy completion. This is the only place where the code exploits the previous combinatorial solution directly. The stored heavy-line set is the first one that reaches the best gain finally retained by the solver: `submit_candidate()` overwrites `best_choice_` only on a strict gain improvement, so later equal-gain candidates do not replace the exported warm start.

Second, the solver carries a dual seed for the Lagrangian bound. The field `warm_dual_seed` is not the final dual vector of the previous search. The code captures it as follows: at the conclusion of any Lagrangian evaluation, the helper `store_best_dual()` stores the current best dual vector into `initial_dual_seed_` if and only if `root_dual_seed_captured_` is still `false`; the flag is then set so that subsequent evaluations do not overwrite the captured seed. The solver then reuses that one-shot seed in three places:

- as the starting point for all subsequent Lagrangian evaluations in the same solve;
- as the initial dual vector for freshly prepared frontier tasks and worker contexts; and
- as the exported seed `warm_dual_seed` for the next value of N , returned via `out.dual_seed = initial_dual_seed_`.

This distinction matters. The witness and the warm heavy-line set come from the last successful exact solve as a whole, whereas the exported dual seed comes from the first completed bound computation inside that solve. The helper `solve_exact()` writes the returned heavy lines back into `warm_start_solution` and writes back a nonempty exported dual seed before it returns, regardless of timeout status. In the actual driver, however, only a non-timed-out solve can affect later steps: on success it rebuilds the witness from the returned heavy-line set and sets `witness_ready=true`, whereas on timeout it prints `[stopped]` and terminates the sweep immediately, so the post-timeout warm buffers are never reused.

5 Lower Bound

5.1 The Root Cover Inequality

I first state the combinatorial lower bound that `cover_inequality_lb()` computes at the root of every witness miss.

Definition 5.1. Let $S \subseteq [N]$. I call S 2-cover-free if every active heavy line in A_N contains at most two points of S .

Proposition 5.2. If $S \subseteq [N]$ is 2-cover-free, then every line cover of S_N uses at least $\lceil |S|/2 \rceil$ lines.

Proof. Any non-heavy line contains at most two points of S_N , hence at most two points of S . By definition, every active heavy line also contains at most two points of S . Therefore every line in any cover accounts for at most two points of S , so at least $\lceil |S|/2 \rceil$ lines are necessary. $\square$

The implementation does *not* solve the maximum-cardinality 2-cover-free problem optimally. It starts from $S = [N]$ and repeatedly deletes the point p with the largest *violation count*—the number of currently active heavy lines in A_N that contain p and still cover at least three points of the current S . When no active heavy line covers three or more points of S , the set S is 2-cover-free and the routine returns $\lceil |S|/2 \rceil$.

5.2 Lagrangian Upper Bound on Additional Gain

At a search node T , I write $P(T)$ for the productive heavy lines and

$$A_T(h) := \text{pts}(h) \cap U(T), \quad r_T(h) := |A_T(h)| - 2$$

for each $h \in P(T)$. The code only materializes active points that lie on at least one productive line, so I define

$$U^+(T) := \bigcup_{h \in P(T)} A_T(h).$$

For a vector $y = (y_i)_{i \in U^+(T)}$ with $0 \leq y_i \leq 1$, the solver evaluates

$$L_T(y) := \sum_{i \in U^+(T)} y_i + \sum_{h \in P(T)} \max \left(0, r_T(h) - \sum_{i \in A_T(h)} y_i \right). \quad (1)$$

Proposition 5.3. For every node T and every y with $0 \leq y_i \leq 1$,

$$\text{current_gain}(T) + L_T(y)$$

is an upper bound on the total gain attainable below T .

Proof. I fix any continuation below T and list the selected heavy lines in the order in which the solver chooses them: $h_1, \dots, h_m$. For each t , let $C_t \subseteq A_T(h_t)$ be the points that are first covered when the solver selects h_t . Then the sets C_t are pairwise disjoint and the gain contributed by h_t is $|C_t| - 2$.

Now fix t and write $O_t = A_T(h_t) \setminus C_t$ for the points of h_t that were already covered earlier in the continuation. Since $|A_T(h_t)| = |C_t| + |O_t|$, I have

$$|C_t| - 2 = r_T(h_t) - |O_t|.$$

I compare this with the h_t -term in (1).

If

$$r_T(h_t) - \sum_{i \in A_T(h_t)} y_i > 0,$$

then

$$\max\left(0, r_T(h_t) - \sum_{i \in A_T(h_t)} y_i\right) + \sum_{i \in C_t} y_i = r_T(h_t) - \sum_{i \in O_t} y_i \geq r_T(h_t) - |O_t| = |C_t| - 2,$$

because each $y_i \leq 1$.

If instead

$$r_T(h_t) - \sum_{i \in A_T(h_t)} y_i \leq 0,$$

then

$$\sum_{i \in C_t} y_i = \sum_{i \in A_T(h_t)} y_i - \sum_{i \in O_t} y_i \geq r_T(h_t) - |O_t| = |C_t| - 2,$$

again because $\sum_{i \in O_t} y_i \leq |O_t|$.

So in both cases

$$|C_t| - 2 \leq \max\left(0, r_T(h_t) - \sum_{i \in A_T(h_t)} y_i\right) + \sum_{i \in C_t} y_i.$$

I sum this inequality over t . The sets C_t are disjoint, so

$$\sum_{t=1}^m \sum_{i \in C_t} y_i \leq \sum_{i \in U^+(T)} y_i.$$

Therefore the additional gain of the continuation is at most $L_T(y)$, and the total gain is at most $\text{current_gain}(T) + L_T(y)$. $\square$

At the root of an exact solve, the program converts the first captured gain bound into a cost bound. If I write that root gain bound as Γ , then the code records

$$\text{lb}_{\text{raw}} = \left\lceil \frac{N - \lfloor \Gamma + 10^{-9} \rfloor}{2} \right\rceil.$$

The small additive tolerance 10^{-9} inside the floor ensures that gain bounds that are integrally exact up to floating-point error are treated as integers rather than rounded down incorrectly. The final root lower bound is

$$\text{lb} = \max\{\text{lb0_floor}, \text{lb}_{\text{cov}}, \text{lb}_{\text{raw}}\}.$$

The source captures that root bound only once, namely at the first call to `capture_root_cost_lb()`; subsequent Lagrangian evaluations do not overwrite it.

5.3 Subgradient Descent and Coordinate Polish

The implementation minimizes (1) by projected subgradient steps. One can present the same dual information as an ascent on the negated functional, but the code itself works directly with the upper-bound functional (1) and therefore performs projected subgradient *descent*. For a fixed y , the line slack is

$$\sigma_h(y) := r_T(h) - \sum_{i \in A_T(h)} y_i.$$

The subgradient coordinate at a productive point i is

$$g_i = 1 - \#\{h \in P(T) : \sigma_h(y) > 0, i \in A_T(h)\}.$$

If the current incumbent cost is C , then any strictly better cover must reach gain at least

$$G_{\text{need}} = N - 2C + 2.$$

At node T , the remaining gain requirement is

$$\tau_T = \max(0, G_{\text{need}} - \text{current_gain}(T)).$$

With this notation, the code uses the step

$$\alpha = \frac{\sigma_{\text{step}} \max(0, L_T(y) - \tau_T)}{\|g\|_2^2},$$

where

$$\sigma_{\text{step}} = \begin{cases} 1.75, & \text{if the node's dual is warm-started, } C \geq 113, \text{ and } |\text{current_choice}| \geq 4, \\ 1.5, & \text{if the node's dual is warm-started but the previous condition fails,} \\ 1.35, & \text{otherwise.} \end{cases}$$

The iteration budget is exactly

$$\min\left(64, 8 + \left\lfloor \frac{|U(T)|}{8} \right\rfloor\right).$$

The initialization is also exact. If a dual seed is available, the code clamps each seed coordinate to $[0, 1]$ and starts there. Otherwise it initializes from scratch: for each productive line h at node T with residual coverage $k = c_T(h)$, it computes the density

$$\frac{k - 2}{k}$$

and sets each active point's initial dual value to the *maximum* density over all productive lines through that point, finally clipping to 1. This initialization is implemented exactly as described: $\text{rem2} = k - 2$, $\text{rem} = k$, $\text{density} = \text{rem2}/\text{rem}$, and the per-point maximum is taken with a conditional assignment.

The implementation adds two short-cut rules on the difficult tail:

- If the node's dual is warm-started, the incumbent cost is at least 112, and $|\text{current_choice}| \geq 4$, then the solver first runs only 16 subgradient iterations.
- It continues to the full budget only if the staged total gain bound lies within 1.0 of the pruning threshold.

After subgradient descent, the code may perform coordinate polish. It does so only when the current total gain bound lies within 0.5 of the pruning threshold. In the current implementation, the coordinate polish runs at most one sweep per Lagrangian evaluation: the constant `kCoordinatePolishMaxSweeps` is computed as `scratch.dual_ready ? 1 : 3`, but `scratch.dual_ready` is set unconditionally to true before the polish phase is reached, so the three-sweep path is not exercised by the current code. For one coordinate y_i , with all other coordinates fixed, the objective has the form

$$y_i + \sum_{h \ni i} \max(0, a_h - y_i),$$

where the constants a_h are the adjusted incident slacks. The slope changes only at the positive values of a_h , so the minimizer is the second-largest positive adjusted slack, clamped to $[0, 1]$. That is exactly the value that the source assigns.

5.4 Branch-Specific Bound Adjustments

After the solver chooses a branch line b , it computes separate gain caps for the include and exclude children. In this subsection I write

$$\Gamma := \text{current_gain}(T) + L_T(y)$$

for the parent total gain bound supplied by [theorem 5.3](#).

For the exclude child, the solver begins with the parent bound and removes the positive slack of b itself. It then inspects each active point p on b . If another productive line through p has positive slack at most $1 - y_p$, then that line could consume part of the residual dual mass at p . Since only one such competing line can consume the same residual mass at that point, the code subtracts the sum of those candidate slacks except for the largest one. Formally, if

$$R_p(b) = \{h \in P(T) \setminus \{b\} : 0 < \sigma_h(y) \leq 1 - y_p, p \in A_T(h)\},$$

then the exclude cap equals

$$\Gamma - \max(0, \sigma_b(y)) - \sum_{p \in A_T(b)} \left(\sum_{h \in R_p(b)} \sigma_h(y) - \max_{h \in R_p(b)} \sigma_h(y) \right),$$

clipped below by $\text{current_gain}(T)$.

For the include child, the solver first credits the definite gain

$$\delta_T(b) = |A_T(b)| - 2.$$

It then removes, for every productive competitor $h \neq b$, the residual dual masses $1 - y_p$ at the active points $p \in A_T(b) \cap A_T(h)$. If I write

$$\rho_h(b) := \sum_{p \in A_T(b) \cap A_T(h)} (1 - y_p),$$

then the include cap is

$$\Gamma + \min(0, \sigma_b(y)) + \sum_{h \in P(T) \setminus \{b\}} (\max(0, \sigma_h(y) - \rho_h(b)) - \max(0, \sigma_h(y))),$$

clipped below by $\text{current_gain}(T) + \delta_T(b)$.

Proposition 5.4. *Both child caps are valid upper bounds on the total gain attainable in the corresponding child.*

Proof. I start from the valid parent bound in [theorem 5.3](#). Write

$$\Gamma = \text{current_gain}(T) + \sum_{i \in U^+(T)} y_i + \sum_{h \in P(T)} \max(0, \sigma_h(y)).$$

I first treat the include child. Selecting b contributes the definite gain

$$\delta_T(b) = |A_T(b)| - 2 = r_T(b).$$

It also removes every point of $A_T(b)$ from the future active set. Hence the global point term loses $\sum_{p \in A_T(b)} y_p$. For each productive competitor $h \neq b$, every shared active point $p \in A_T(b) \cap A_T(h)$ reduces the residual line term by exactly

$$1 - y_p,$$

because $r_T(h)$ drops by 1 and the corresponding y_p leaves the active-point sum. Therefore the competitor slack changes from $\sigma_h(y)$ to $\sigma_h(y) - \rho_h(b)$, where

$$\rho_h(b) = \sum_{p \in A_T(b) \cap A_T(h)} (1 - y_p).$$

So the same dual vector y gives the include child the bound

$$\text{current_gain}(T) + \delta_T(b) + \sum_{i \in U^+(T) \setminus A_T(b)} y_i + \sum_{h \in P(T) \setminus \{b\}} \max(0, \sigma_h(y) - \rho_h(b)).$$

Subtracting the parent expression Γ and using

$$\delta_T(b) - \sum_{p \in A_T(b)} y_p = r_T(b) - \sum_{p \in A_T(b)} y_p = \sigma_b(y)$$

shows that the change equals

$$\min(0, \sigma_b(y)) + \sum_{h \in P(T) \setminus \{b\}} (\max(0, \sigma_h(y) - \rho_h(b)) - \max(0, \sigma_h(y))),$$

which is exactly the coded include cap before the final clipping below by $\text{current_gain}(T) + \delta_T(b)$. That clipping only strengthens the upper bound.

I now treat the exclude child. Line b is forbidden, so the parent term $\max(0, \sigma_b(y))$ disappears immediately. Fix one active point $p \in A_T(b)$ and consider the set $R_p(b)$ from the statement. Because two distinct geometric lines share at most one point, every competitor line that meets b appears in at most one such set $R_p(b)$, so deductions made at different points of b do not double count any line.

Now fix any feasible continuation in the exclude child. Among the selected lines from $R_p(b)$, at most one can cover p first. Every other selected line $h \in R_p(b)$ has $p \in O_h$ in the notation of [theorem 5.3](#). For such a line,

$$\max(0, \sigma_h(y)) + \sum_{i \in C_h} y_i - (|C_h| - 2) = \sum_{q \in O_h} (1 - y_q) \geq 1 - y_p \geq \sigma_h(y),$$

because $h \in R_p(b)$ implies $0 < \sigma_h(y) \leq 1 - y_p$. So for every selected line in $R_p(b)$ except possibly the first one that covers p , the parent bound overestimates the line's realizable gain contribution by at least its entire positive slack. If S_p is the subset of $R_p(b)$ that the continuation actually selects and $S_p \neq \emptyset$, then I may therefore subtract at least

$$\sum_{h \in S_p} \sigma_h(y) - \max_{h \in S_p} \sigma_h(y).$$

If $S_p = \emptyset$, the deduction is 0. When $S_p \neq \emptyset$, since $S_p \subseteq R_p(b)$ and all slacks are nonnegative,

$$\sum_{h \in S_p} \sigma_h(y) - \max_{h \in S_p} \sigma_h(y) \leq \sum_{h \in R_p(b)} \sigma_h(y) - \max_{h \in R_p(b)} \sigma_h(y).$$

Summing this deduction over the points $p \in A_T(b)$ and also removing the forbidden contribution $\max(0, \sigma_b(y))$ yields exactly the coded exclude cap. The final clipping below by $\text{current_gain}(T)$ is again only a strengthening. $\square$

The supplied full-sweep log shows that, on the 16 non-witness rows in the new block, the root cover bound satisfies

$$78 \leq \text{lb}_{\text{cov}} \leq 83$$

and the root Lagrangian cost bound satisfies

$$111 \leq \text{lb}_{\text{raw}} \leq 115.$$

On those same 16 rows the root diagnostic gap $\text{ub} - \text{lb}$ is always exactly 1. The carried floor lb_{floor} is always larger, ranging from 123 to 126, so it dominates the final root lower bound throughout the block. On the 11 mode-D rows, the same log records

$$162,178,416 \leq \text{lag_iters} \leq 3,565,213,480,$$

$$2,193,207 \leq \text{lag_prune} \leq 46,642,385,$$

strong branching counts between 11 and 17, and maximum search depth 117.

6 Correctness and Optimality

Lemma 6.1 (Monotonicity). *For every $N \geq 1$, I have $f(N + 1) \geq f(N)$.*

Proof. Any cover of $\mathcal{S}_{N+1}$ restricts to a cover of $\mathcal{S}_N$, because $\mathcal{S}_N \subseteq \mathcal{S}_{N+1}$. Therefore the optimum cannot decrease when I add a point. $\square$

6.1 The Exclusive Dependency Rule

Definition 6.2. *For a productive line h at node T , I define*

$$E_T(h) := \{i \in U(T) : h \text{ is the unique productive line through } i\}.$$

Theorem 6.3 (Exclusive Dependency Rule). *If $|E_T(h)| \geq 3$, then some optimal completion below T contains h .*

Proof. I fix an optimal completion below T that omits h . Every point of $E_T(h)$ must then be covered by lines that are not productive at T . Any such line contains at most two active points of $U(T)$, so if it covers at least one point of $E_T(h)$, it covers at most one active point outside $E_T(h)$.

Let k be the number of lines in the completion that cover at least one point of $E_T(h)$, and let d be the number of active points outside $E_T(h)$ that those k lines cover. The previous paragraph gives $d \leq k$. I replace those k lines by the single line h and then cover the displaced d outside points with at most $\lceil d/2 \rceil \leq \lceil k/2 \rceil$ additional lines. The total cost changes by at most

$$\Delta(k) := -k + 1 + \left\lceil \frac{k}{2} \right\rceil.$$

For $k = 2$, $\Delta(2) = 0$. For $k = 3$, $\Delta(3) = 0$. For $k \geq 4$, $\Delta(k) \leq -k/2 + 1.5 < 0$.

Because $|E_T(h)| \geq 3$, at least $\lceil 3/2 \rceil = 2$ non-productive lines are needed to cover $E_T(h)$ without h , so $k \geq 2$. Hence $\Delta(k) \leq 0$ for all $k \geq 2$, and the replacement never increases the cost. I have produced an optimal completion that contains h . $\square$

6.2 Frontier Dominance

Lemma 6.4 (Frontier dominance). *Suppose two tasks T_1 and T_2 satisfy*

$$U(T_1) = U(T_2), \quad B(T_1) = B(T_2),$$

and

$$\text{current_gain}(T_1) \geq \text{current_gain}(T_2).$$

Then discarding T_2 cannot remove an optimal solution.

Proof. The active point set determines the residual coverage of every heavy line. The set $B(T)$ records exactly which unavailable lines still have residual coverage at least 3. Any line with residual coverage at most 2 is irrelevant to future gain, because it can never become productive again when the active set only shrinks. Therefore T_1 and T_2 have exactly the same future productive options and exactly the same future gain opportunities.

Every continuation available below T_2 is therefore also available below T_1 , with the same additional gain from that point onward. Since T_1 starts with at least as much accumulated gain, it dominates T_2 . $\square$

6.3 Exact Solve and the Three Modes

Theorem 6.5. *If a call to `solve_exact()` terminates without hitting the per- N time limit, then the returned answer is optimal.*

Proof. The solver searches over include/exclude decisions for productive heavy lines. By [theorem 2.6](#), once no productive line remains, the current gain already determines the optimal final cover size for that branch, because every remaining point can be closed with pair lines and at most one singleton line.

Every transformation that the solver applies before that terminal case is safe. The Exclusive Dependency Rule is safe by [theorem 6.3](#). Frontier-state deletion is safe by [theorem 6.4](#). Root and node pruning by the Lagrangian bound are safe by [theorem 5.3](#). The include and exclude child caps are safe by [theorem 5.4](#). The frontier builder only reorganizes the same search tree into independent tasks; it does not discard any task except by those already proved safe rules.

Consequently, if the solver terminates without timeout, then it has exhausted every branch whose bound can still beat the incumbent. The incumbent is therefore globally optimal. $\square$

Proposition 6.6 (Mode W). *If the driver reports mode W at N , then the reported answer is $f(N)$.*

Proof. The current witness was built after an earlier successful exact solve, say at step N' . By [theorem 4.1](#) and [theorem 2.6](#), the witness has exactly $f(N') = \text{prev}$ lines and covers all of $\mathcal{S}_{N'}$. All sweep steps between N' and N fired mode W, so the witness covered each of the intermediate points as well, because `witness_covers_point` returned true at every such step. In mode W the witness covers (N, p_N) , so it covers all of $\mathcal{S}_N$ with prev lines. Hence $f(N) \leq \text{prev}$. On the other hand, [theorem 6.1](#) gives $f(N) \geq f(N-1) = \text{prev}$. Therefore $f(N) = \text{prev}$, which is exactly what the driver reports. $\square$

Proposition 6.7 (Mode R). *If the driver reports mode R at N , then the reported answer is $f(N)$.*

Proof. Mode R means that the witness missed, the driver called `solve_exact()`, the call terminated without timeout, and the returned DFS node count was zero. The value of the answer does not depend on that node count. By [theorem 6.5](#), every non-timed-out exact solve returns the optimum, hence the mode-R answer is $f(N)$. $\square$

Proposition 6.8 (Mode D). *If the driver reports mode D at N , then the reported answer is $f(N)$.*

Proof. Mode D means that the witness missed, the driver called `solve_exact()`, the call terminated without timeout, and the returned DFS node count was positive. Again the node count is irrelevant to correctness. [Theorem 6.5](#) gives the conclusion directly. $\square$

Remark 6.9. The logged fields `lb`, `lb_raw`, `gap`, and `gap_raw` are root diagnostics, not final global gaps. In the preserved block $N = 862, \dots, 882$, every non-witness row has root gap 1, yet the solver certifies those rows because the later exact search closes the remaining global gap to zero before termination.

Corollary 6.10 (Certification policy). *Whenever the program prints an answer line for some N , that answer is certified optimal. If the per- N time limit fires first, the program prints a `[stopped]` message and does not report an uncertified answer for that N .*

Proof. Modes W, R, and Dare exhaustive, and [theorems 6.6](#) to [6.8](#) prove optimality in each case. If a timeout occurs, the driver breaks immediately after printing `[stopped]` and before printing any answer line for that value of N . $\square$

7 OEIS Sequence and Data

OEIS entry A373813 [\[1\]](#) records the minimum line cover numbers for the prime-point sets S_N ; [figure 5](#) shows the complete staircase over $N = 1, \dots, 882$. This work extends it by 21 new terms ($N = 862$ through $N = 882$), including three new awkward primes—primes whose arrival forces the minimum line count to increase. In this new block the cover size increases at exactly three indices:

$$864, \quad 871, \quad 875.$$

These are the new awkward primes.

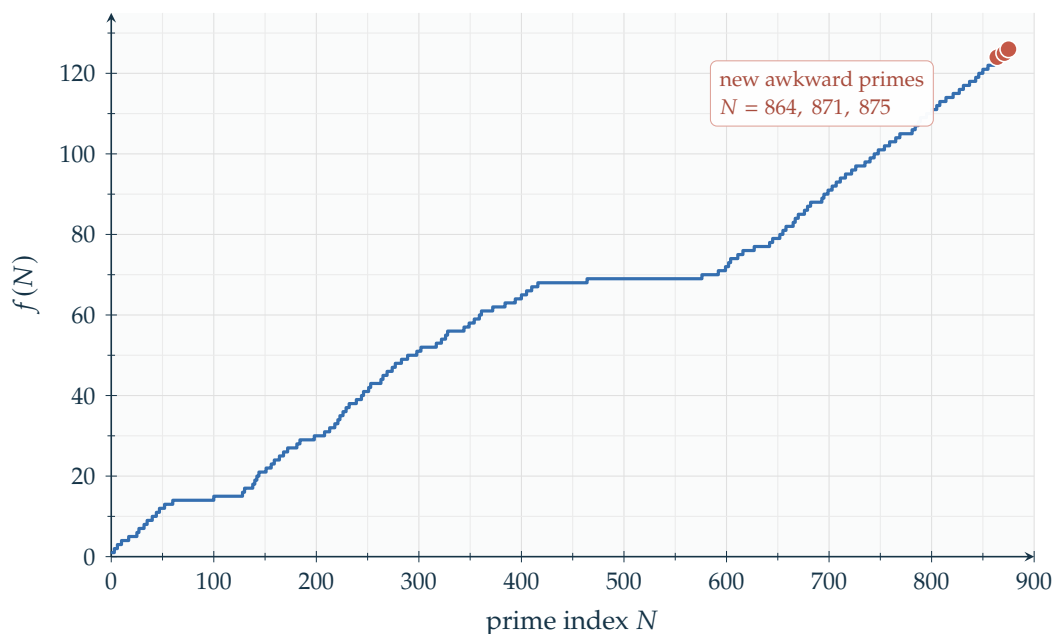


Figure 5: The minimum line cover number $f(N)$ plotted as a staircase over all certified values $N = 1, \dots, 882$. Each vertical step marks an *awkward prime*: the index at which the optimum first increases. The three filled coral marks identify the new awkward primes reported here. The sequence is OEIS A373813; the current record is $f(882) = 126$.

I report the new record $N = 882$, certified optimal. In the preserved sweep log `PRIMECOVER_WR_882.txt`, the cumulative wall-clock time through $N = 861$ (the prior record

boundary) is 13,115.300 seconds, and the cumulative time through $N = 882$ is 51,256.102 seconds, both on a standard consumer laptop. The isolated row for $N = 882$ has per-instance time 1,092.070 seconds—under three hours. The cumulative times are derived from the completed rows of the log rather than from a final `total` time line, because the log ends with an interrupted live-display line at $N = 883$.

The first table lists the 21 new terms together with the row fields that the preserved log prints for every mode. Witness rows bypass exact search, so on those rows the log writes placeholders `lb_cov=0`, `lb=lb_raw=prev`, and `nodes=0`. The second table isolates the mode-D rows and records the search-intensive diagnostics that are meaningful only when DFS actually runs. The current source prints several later fields, including `frontier`, `warm_size`, `greedy_heavy`, `forced_root`, `polish_sweeps`, and `root_ub_frac`; the preserved sweep log predates those additions, so I do not reconstruct them retrospectively.

Table 1: The 21 new OEIS terms, with per-row statistics extracted from `PRIMECOVER_WR_882.txt`. The column `time (s)` is the per-instance wall-clock time printed on that row.

N	p_N	$f(N)$	mode	time (s)	active	ub0	lb_floor	lb_cov	lb	gap	ub_raw	lb_raw	gap_raw	nodes
862	6689	123	R	0.016016	9254	124	123	78	123	1	129	112	17	0
863	6691	123	D	488.028	9271	124	123	81	123	1	130	112	18	7324780
864	6701	124	D	4719.479	9291	124	123	80	123	1	131	112	19	37990028
865	6703	124	W	0.010935	9314	124	124	0	124	0	131	124	7	0
866	6709	124	R	0.014763	9336	125	124	83	124	1	131	112	19	0
867	6719	124	D	345.253	9366	125	124	80	124	1	129	111	18	6690029
868	6733	124	D	287.188	9377	125	124	80	124	1	131	112	19	5182435
869	6737	124	W	0.010331	9400	124	124	0	124	0	131	124	7	0
870	6761	124	D	1164.335	9414	125	124	82	124	1	131	112	19	16058076
871	6763	125	D	5354.545	9435	125	124	81	124	1	132	113	19	42645634
872	6779	125	R	0.015590	9456	126	125	80	125	1	132	113	19	0
873	6781	125	D	770.716	9473	126	125	78	125	1	132	113	19	12566060
874	6791	125	D	12683.017	9495	126	125	81	125	1	132	114	18	109259523
875	6793	126	D	10741.622	9522	126	125	81	125	1	133	113	20	83618987
876	6803	126	R	0.015696	9548	127	126	82	126	1	133	113	20	0
877	6823	126	D	494.414	9566	127	126	83	126	1	133	113	20	9037019
878	6827	126	R	0.016462	9584	127	126	80	126	1	134	114	20	0
879	6829	126	W	0.011755	9603	126	126	0	126	0	134	126	8	0
880	6833	126	W	0.011618	9629	126	126	0	126	0	134	126	8	0
881	6841	126	W	0.011341	9651	126	126	0	126	0	134	126	8	0
882	6857	126	D	1092.070	9673	127	126	80	126	1	134	115	19	16011317

Table 2: Mode-D rows from the same block. Here `prod` and `pinc` are the productive-line count and the productive incidence count at the exact-solve root after the initial root forcing pass.

N	prod	pinc	lag_iters	lag_prune	strong_branch	depth	nodes
863	9271	31142	236585317	3133257	12	104	7324780
864	9291	31206	1275331851	16148250	17	104	37990028
867	9366	31447	205654995	2825657	14	115	6690029
868	9377	31489	162178416	2193207	16	106	5182435
870	9414	31611	529171152	6870068	11	102	16058076
871	9435	31680	1444293480	18244100	11	87	42645634
873	9473	31801	398551188	5353238	14	104	12566060
874	9495	31870	3565213480	46642385	13	108	109259523
875	9522	31960	2792185969	35627455	11	99	83618987
877	9566	32107	281706768	3850015	14	117	9037019
882	9673	32464	513965847	6873216	14	115	16011317

The new block is heavily incremental. Five rows are witness hits, five rows close before the DFS node counter rises above zero, and only eleven rows require full depth-first search. Within the preserved sweep log, the hardest row is $N = 874$, which reaches 109,259,523 DFS nodes and 12,683.017 seconds. The final new term, $N = 882$, is certified at 126 lines with a per-instance time of 1,092.070 seconds.

8 Code Availability

The solver is a single C++23 source file, `primecover1024.cpp`. The suffix “1024” refers first to the fixed bitset capacity `kBitCapacity = 1024`, which is the hard limit for the

active-point masks used throughout the exact search, and second to the matching default execution cap `kExecutionLimit = 1024`. The program accepts a command-line horizon `requested_n`, but it clamps that horizon to at most 1024, so the filename is a literal statement about the instance-size ceiling of this implementation. In the current source, `kStartN = 1` and `kPerNTimeLimitSeconds = 0`, so the default build starts from $N = 1$ and disables the watchdog timeout. A repository release should archive both the current source and the preserved record log, since the source now prints a few diagnostics that the older `PRIMECOVER_WR_882.txt` file does not contain.

References

References

- [1] OEIS Foundation Inc., *The On-Line Encyclopedia of Integer Sequences*, A373813, <https://oeis.org/A373813>.
- [2] Placeholder entry for the prior $N = 861$ record.
- [3] A. M. Geoffrion, Lagrangean relaxation for integer programming, *Mathematical Programming Study* 2 (1974), 82–114.
- [4] M. Held, P. Wolfe, and H. P. Crowder, Validation of subgradient optimization, *Mathematical Programming* 6 (1974), 62–88.
- [5] M. L. Fisher, The Lagrangian relaxation method for solving integer programming problems, *Management Science* 27 (1981), no. 1, 1–18.